AWS SAA-C03 Study Guide

# Module 3a
# Storage Deep Dive

Object vs block vs file • S3 internals, security & storage classes • EBS & instance store • EFS & the FSx family • Hybrid storage • Migration services • Worked practice questions

# 1. The Three Storage Paradigms (know these cold)

| | Object (S3) | Block (EBS, instance store) | File (EFS, FSx) |
|---|---|---|---|
| Unit | Whole objects (file + metadata + key) via HTTPS API | Raw disk blocks presented as a volume | Files/folders over a network protocol (NFS/SMB/Lustre) |
| Access | GET/PUT the entire object (no in-place partial edits) | OS formats it with a filesystem; random read/write any byte | Many clients mount the same share simultaneously |
| Scale | Effectively unlimited; pay for what's stored | Fixed provisioned size per volume; one instance at a time (few exceptions) | Elastic (EFS) or provisioned (FSx); shared |
| Best for | Static assets, backups, data lakes, logs, media, anything web-scale | Databases, boot volumes, low-latency single-instance workloads | Shared content, home dirs, lift-and-shift apps expecting a filesystem |

**CORE CONCEPT — The #1 storage question pattern:** Match the workload's ACCESS PATTERN to the paradigm first, then pick the product. 'Hundreds of instances need shared access' → file (EFS/FSx). 'Database needs low-latency disk' → block (EBS). 'Serve images to millions / archive logs' → object (S3). Only after that do cost/performance tiers matter.

# 2. Amazon S3 In Depth

## 2.1 The mental model

S3 stores **objects** (up to 5 TB) in **buckets**. A bucket has a globally unique name but lives in ONE Region; objects are addressed by **key** — there are no real folders, just key prefixes that consoles render as folders ('logs/2026/08/app.log'). Durability is **11 nines** (99.999999999%): objects are stored redundantly across ≥3 AZs (expect to lose 1 object in 10 million every 10,000 years) — durability is about not LOSING data; availability (99.9–99.99% by class) is about reaching it right now. S3 has **strong read-after-write consistency**: any read after a successful write returns the new data — older courses' 'eventual consistency' caveats are obsolete.

## 2.2 Moving data in and out efficiently

- **Multipart upload:** split large objects into parts uploaded in parallel (retry only failed parts). Required >5 GB, recommended >100 MB.
- **Transfer Acceleration:** upload to the nearest edge location, then ride AWS's backbone to the bucket's Region — for long-distance uploads (global users → one bucket).
- **Byte-range fetches:** parallel partial GETs for big downloads (also lets you read just a file's header).
- **Performance:** 3,500 writes / 5,500 reads per second PER PREFIX, and prefixes are unlimited — spread keys across prefixes to scale (date-stamped prefixes, hash prefixes).
- **Event notifications:** object created/removed events → Lambda, SQS, SNS, or (all events, with filtering and multiple targets) EventBridge — the glue for 'process every uploaded file' pipelines.

## 2.3 Versioning, replication, and object protection

- **Versioning** (bucket-level, on/suspend — never off again): every overwrite creates a new version; a DELETE just adds a 'delete marker' — remove the marker and the object is back. Protects against accidental deletion/overwrite. Prerequisite for replication and MFA Delete (MFA required to permanently delete versions).
- **Replication:** asynchronous copy of new objects to another bucket — **CRR** (cross-Region: DR, compliance, latency to remote users) or **SRR** (same-Region: log aggregation into one bucket, prod→test copies). Needs versioning on both sides. Existing objects need S3 Batch Replication. Delete markers optionally replicated; permanent version deletes never are.

- **Object Lock (WORM):** versioned buckets only. **Compliance mode**: NOBODY, not even root, can delete/shorten retention — the regulatory answer. **Governance mode**: privileged IAM users can override. **Legal hold**: indefinite, on/off per object. 'Data must be immutable/undeletable for N years' → Object Lock Compliance.

## 2.4 S3 security layers (outside-in)

- **Block Public Access:** account- and bucket-level master switches that override any policy/ACL trying to make data public. On by default; the first thing to check in 'bucket accidentally public' questions.
- **Bucket policies:** resource-based JSON on the bucket — grant cross-account access, enforce conditions (aws:SecureTransport for TLS-only, aws:PrincipalOrgID for org-only, require SSE headers). The main tool for bucket-wide rules.
- **ACLs:** legacy per-object grants — AWS recommends disabling (Object Ownership: bucket owner enforced). Only correct in niche legacy scenarios.
- **Presigned URLs:** a time-limited URL carrying the SIGNER's permissions — let a user download/upload one private object without credentials. 'Share a private file for 24 hours' / 'let users upload directly to S3' → presigned URL.
- **Access Points:** named endpoints each with their own policy — manage per-application access to one huge shared bucket without a monster bucket policy. **Object Lambda:** transform data on read (redact PII for one consumer) without storing copies.
- **Static website hosting** serves a bucket over HTTP only; the production pattern is CloudFront in front with **Origin Access Control (OAC)** so the bucket stays private and only CloudFront can read it (HTTPS, caching, custom domain via ACM).

## 2.5 Storage classes — the cost/latency dial

| Class | Retrieval | Min store | Pick when |
|---|---|---|---|
| Standard | ms | — | Hot data; ≥3 AZ; no retrieval fee |
| Intelligent-Tiering | ms | — | UNKNOWN/shifting patterns — auto-moves objects between tiers (incl. optional archive tiers); tiny monitoring fee; NO retrieval fees; never guess wrong |
| Standard-IA | ms | 30 d | Known-infrequent but must be instant (backups, DR files); cheaper storage, per-GB retrieval fee |
| One Zone-IA | ms | 30 d | Infrequent AND re-creatable (thumbnails, replicas) — ~20% cheaper, ONE AZ (data lost if that AZ is destroyed) |
| Glacier Instant Retrieval | ms | 90 d | Archive touched ~quarterly needing millisecond reads (medical images, news archives) |
| Glacier Flexible Retrieval | Expedited 1–5 min / Standard 3–5 h / Bulk 5–12 h (free) | 90 d | True archive; minutes–hours retrieval acceptable |
| Glacier Deep Archive | Standard 12 h / Bulk 48 h | 180 d | Cheapest storage in AWS; 7–10-year retention, retrieval in hours–days |

- **Lifecycle rules** automate transitions (Standard → IA after 30 d → Glacier after 90 d) and expirations (delete after 7 y; abort incomplete multipart uploads — a sneaky cost saver). Transitions only move DOWN the ladder. Deleting early (before min duration) still bills the minimum.
- Gotchas: objects <128 KB don't auto-tier in Intelligent-Tiering and transition poorly (per-object overhead); retrieval fees can exceed savings if 'infrequent' turns out frequent — that's exactly why Intelligent-Tiering exists.

**EXAM TIP:** 'Access pattern unknown/unpredictable' → Intelligent-Tiering, always. 'Compliance archive, cheapest, rarely if ever read' → Deep Archive. 'Infrequent but re-creatable' → One Zone-IA. 'Immediately available backup copies at lower cost' → Standard-IA.

# 3. EBS & Instance Store: Block Storage

## 3.1 How EBS works

An EBS volume is network-attached block storage living in ONE AZ, replicated within that AZ. It attaches to one instance (in the same AZ) at a time — the instance sees a raw disk. Because it's network-attached: it persists independently of the instance (survives stop/terminate if configured), you can detach/reattach to another instance, and performance is bounded by both the VOLUME's provisioned IOPS/throughput AND the INSTANCE's EBS bandwidth — a max-IOPS volume on a small instance is throttled by the instance. Moving a volume across AZ/Region = snapshot it, then create a new volume from the snapshot in the target location.

## 3.2 Volume types — decide by IOPS, throughput, and boot

| Type | Specs to remember | Choose when |
|---|---|---|
| gp3 (SSD) | Baseline 3,000 IOPS & 125 MB/s at ANY size; provision to 16,000 IOPS / 1,000 MB/s independently of size; ~20% cheaper than gp2 | The default for everything: boot, general workloads, most DBs. 'gp2 volume needs more IOPS without growing it' → migrate to gp3 |
| gp2 (SSD) | 3 IOPS per GB, burst to 3,000; max 16,000 (needs 5.3 TB) | Legacy — appears as the 'wrong current choice' in questions |
| io1/io2 (SSD) | Provisioned IOPS up to 64,000 (256,000 on io2 Block Express); sub-ms; io2 = 99.999% durability; supports MULTI-ATTACH (up to 16 Nitro instances, cluster-aware filesystem required) | I/O-hungry databases, >16,000 IOPS, latency-critical, or multi-attach clustering |
| st1 (HDD) | Throughput-optimized, up to 500 MB/s; cheap; CANNOT boot | Sequential big-data/log/ETL streams where MB/s matters, not IOPS |
| sc1 (HDD) | Cold, cheapest EBS; CANNOT boot | Rarely-accessed sequential data kept on a volume anyway |

## 3.3 Snapshots and encryption

- **Snapshots** are incremental, block-level backups stored in S3 (you only pay for changed blocks). Copy them across Regions/accounts for DR. Restored volumes lazy-load blocks (first read of each block is slow); **Fast Snapshot Restore** pre-warms for full performance instantly ($$). **Data Lifecycle Manager** automates snapshot schedules/retention. **Recycle Bin** can retain deleted snapshots.
- **Encryption** (KMS, transparent, no performance penalty): set at creation — covers data at rest, data in transit to the instance, and all snapshots/volumes derived from it. Un-encrypted volume → encrypted: snapshot → COPY the snapshot with encryption enabled → create volume. Account setting can force encryption-by-default for all new volumes.

## 3.4 Instance store — the fast, disposable disk

Instance store is storage on disks PHYSICALLY inside the host — NVMe with millions of IOPS, no network hop, included in the instance price. The trade: it's **ephemeral** — data is gone on stop, hibernate, terminate, or hardware failure, and you can't detach/snapshot it. Correct ONLY for data you can lose: caches, buffers, scratch space, or data replicated at the application layer (e.g., a Cassandra node). 'Highest possible storage IOPS' with no durability requirement → instance store, beating even io2.

# 4. EFS & the FSx Family: File Storage

## 4.1 EFS — elastic NFS for Linux

EFS is a Regional, fully managed NFS filesystem: thousands of Linux clients (EC2, ECS, Lambda, on-prem over DX/VPN) mount it simultaneously through **mount targets** (an ENI per AZ, guarded by security groups allowing NFS port 2049). It grows and shrinks automatically — no capacity planning — and spans all AZs by default (Regional class); **One Zone** classes cost less for non-critical data. **Storage tiers** Standard → Infrequent Access → Archive with lifecycle management move cold files automatically (~90% savings). Throughput modes: **Elastic** (default, auto-scales — pick this), Provisioned, Bursting. Performance modes: General Purpose (default) vs Max I/O (massive parallelism, higher latency — only when told thousands of concurrent clients need aggregate throughput). Linux/NFS ONLY — no Windows.

### 4.2 FSx — purpose-built filesystems

| FSx for... | Protocol / world | The one-liner trigger |
|---|---|---|
| Windows File Server | SMB, NTFS, Active Directory integration, DFS namespaces, shadow copies; Multi-AZ option | 'Windows file share', 'AD-integrated storage', 'NTFS permissions' |
| Lustre | POSIX parallel filesystem; 100s of GB/s, millions IOPS, sub-ms; links to S3 (lazy-loads objects as files, writes results back) | 'HPC', 'machine-learning training data', 'video rendering', 'process S3 data as files at extreme speed'. Scratch (temp, cheap) vs Persistent deployment |
| NetApp ONTAP | Multi-protocol NFS + SMB + iSCSI; snapshots, cloning, dedup/compression; SnapMirror replication | 'Both Linux and Windows clients on the same share', 'migrate existing NetApp' |
| OpenZFS | NFS, ZFS features (snapshots, clones), very low latency | 'Migrate a ZFS server' |

**EXAM TIP:** Linux-shared → EFS. Windows/AD → FSx for Windows. HPC/ML speed → FSx for Lustre. Mixed NFS+SMB → FSx for ONTAP. These four sentences answer a surprising number of questions verbatim.

## 5. Hybrid Storage: Storage Gateway

Storage Gateway is a VM (or hardware appliance) on-premises presenting familiar interfaces locally while data lives in AWS — the bridge for datacenters that can't move everything yet. Match the flavor to the interface:

| Flavor | Presents on-prem | Data lives | Scenario |
|---|---|---|---|
| S3 File Gateway | NFS/SMB share | S3 objects (recently used files CACHED locally) | 'On-prem apps write files; store in S3; low-latency access to recent files'; feeds S3 lifecycle/analytics |
| FSx File Gateway | SMB share | FSx for Windows | Low-latency on-prem cache for FSx Windows shares |
| Volume Gateway – Cached | iSCSI block volumes | Primary in S3; hot blocks cached locally | 'Expand on-prem block storage without buying disks' |
| Volume Gateway – Stored | iSCSI block volumes | Primary ON-PREM; async EBS snapshots to AWS | 'Keep all data local (latency/compliance) with cloud backup/DR' |
| Tape Gateway | Virtual tape library (iSCSI VTL) | S3 → Glacier/Deep Archive | 'Replace physical backup tapes; keep existing backup software (Veeam/NetBackup)' |

## 6. Getting Data (and Servers) Into AWS

### 6.1 Online vs offline — do the bandwidth math

Transfer time = data ÷ usable bandwidth. 100 TB over a fully-dedicated 1 Gbps link ≈ 10 days; over 100 Mbps ≈ 3+ months. When the scenario gives you TBs + a link speed + a deadline, compute (roughly) and pick: feasible over the wire → online tools; not feasible (or link needed for production) → Snow family. Rule of thumb: if it takes more than ~a week over the network, ship a box.

### 6.2 The tools

| Service | What it does | Choose when |
|---|---|---|
| DataSync | Managed ONLINE transfer agent: NFS/SMB/HDFS/object ↔ S3/EFS/FSx. Incremental, scheduled, bandwidth-throttled, verified, preserves metadata; ~10x faster than scripted rsync/CLI | One-time or SCHEDULED recurring file migration/sync over network. 'Nightly sync of the NAS to S3' → DataSync (not Storage Gateway — that's ongoing ACCESS, not transfer) |
| Snow family | Ruggedized shipped devices: Snowball Edge Storage Optimized ~80 TB usable (also compute-optimized variants for edge processing); order many for PB scale (Snowmobile retired) | Too much data / too little bandwidth / disconnected sites. Also edge compute in the field |
| Transfer Family | Managed SFTP/FTPS/FTP/AS2 endpoints storing straight into S3 or EFS | 'Partners already upload via SFTP and must not change their process' |
| DMS (+SCT) | Database Migration Service: replicates databases with source ONLINE (minimal downtime), incl. continuous CDC replication. Schema Conversion Tool converts engines (Oracle→Aurora PostgreSQL) | Any DB migration; 'heterogeneous' → add SCT; 'keep source running / near-zero downtime' → DMS with CDC |
| Application Migration Service (MGN) | Agent-based block-level replication of entire servers; cutover launches them as EC2 | Lift-and-shift ('rehost') fleets of VMs/physical servers |

**EXAM TIP:** Disambiguation drill — 'transfer files': recurring network sync = DataSync; keep accessing on-prem afterward = File Gateway; partners push via SFTP = Transfer Family; petabytes/deadline = Snowball; databases = DMS; whole servers = MGN.

## 7. Reference Architecture: Media Asset Pipeline

A photo platform: users worldwide upload originals; the system generates thumbnails; recent images are hot, older ones cool fast; originals must be kept 10 years for legal reasons; on-prem editors need file access to recent work.

- **Ingest:** browser gets a presigned PUT URL from the API → uploads DIRECTLY to S3 (app servers never touch bytes) with Transfer Acceleration for distant users. S3 event → EventBridge → Lambda creates thumbnails into a 'derived' bucket (One Zone-IA — re-creatable).
- **Serving:** CloudFront with OAC in front of both buckets; buckets fully private; Block Public Access on.
- **Cost curve:** originals bucket uses Intelligent-Tiering (unpredictable access) + lifecycle: to Glacier Flexible after 180 d, Deep Archive after 2 y, expire at 10 y. Versioning + Object Lock (Compliance, 10 y) on originals for legal hold; CRR to a second Region for DR.
- **Hybrid:** the editing office mounts an S3 File Gateway share — recent files cached locally at LAN speed, everything durably in S3. The initial 60 TB back-catalog arrived via Snowball; ongoing nightly camera-card dumps sync via DataSync.

Notice how each requirement selected a feature — that's the skill the exam tests. Re-derive this design from the requirements list without looking.

## 8. Worked Practice Questions

**Practice Q1.** A firm must retain transaction records for 7 years. Records are written once, almost never read, and retrieval within 24 hours is acceptable. MOST cost-effective storage?

✗ S3 Standard-IA — instant access you don't need, at ~5x the price.

✗ Glacier Instant Retrieval — pays for millisecond access; unnecessary.

✓ S3 Glacier Deep Archive with a lifecycle rule — cheapest class in AWS; standard retrieval 12 h fits '24 hours'. Match the retrieval SLA to the class, then take the cheapest that qualifies.

**Practice Q2.** A web app on 20 EC2 Linux instances across 3 AZs must read/write the same directory of user documents with POSIX permissions. Which storage?

✗ EBS Multi-Attach — io1/io2 only, max 16 instances, single AZ, needs a cluster filesystem: wrong on three counts.

✗ S3 mounted via tools — object storage isn't POSIX; partial writes/locking break.

✓ EFS mounted on all instances — Regional NFS built exactly for many-client shared POSIX access across AZs. 'Many Linux instances, shared filesystem' → EFS, reflexively.

**Practice Q3.** A database on gp2 needs 12,000 IOPS but only stores 500 GB. The current volume delivers 1,500 IOPS. MOST cost-effective fix?

✗ Grow gp2 to 4 TB for 12,000 IOPS — paying for 3.5 TB you don't need to buy IOPS.

✗ io2 with 12,000 provisioned IOPS — works, but provisioned-IOPS pricing exceeds the alternative.

✓ Migrate to gp3 and provision 12,000 IOPS on the 500 GB volume — gp3 decouples IOPS from size and is cheaper per GB. gp3's independent IOPS is one of the most-tested storage facts.

**Practice Q4.** An HPC genomics workload must process 200 TB currently in S3 with sub-millisecond file access and hundreds of GB/s aggregate throughput, for about a week. Best solution?

✗ EFS Max I/O — strong scale-out, but not S3-linked and not in Lustre's throughput class.

✗ Copy to big gp3 volumes per node — no shared namespace; copy time and cost explode.

✓ FSx for Lustre (scratch deployment) linked to the S3 bucket — lazy-loads objects as POSIX files at extreme speed, writes results back to S3, and scratch mode keeps a temporary week-long cluster cheap. 'HPC + S3 data + filesystem' = Lustre.

**Practice Q5.** A company's 90 TB archive must move to S3. The office link is 50 Mbps and is saturated by daily business. The migration must finish within 3 weeks. What should they use?

✗ DataSync with throttling — 90 TB at even a full 50 Mbps ≈ 5+ months; throttled, worse.

✗ Transfer Acceleration — speeds the last mile to S3, but cannot exceed the 50 Mbps pipe.

✓ Order a Snowball Edge, load locally at LAN speed, ship it — device turnaround comfortably beats 3 weeks and spares the production link. Bandwidth math first; when the wire loses, ship.

**Practice Q6.** Regulators require that uploaded audit logs in S3 cannot be deleted or altered by ANYONE, including administrators, for 5 years. Which configuration?

✗ Bucket policy denying deletes — an admin can edit the policy.

✗ Versioning + MFA Delete — raises the bar but root/authorized users can still permanently delete.

✓ Versioning + S3 Object Lock in COMPLIANCE mode, 5-year retention — immutable against every identity including root until retention lapses. 'No one, not even admins' is the Compliance-mode fingerprint (Governance mode = privileged override allowed).

**Practice Q7.** An on-prem Windows app writes files to a local share. The company wants all files stored durably in S3 while keeping LAN-speed access to recently used files, changing nothing in the app. Which service?

✗ DataSync — moves/syncs data on schedules; doesn't present an ongoing live share backed by S3.

✗ FSx for Windows — durable SMB in AWS, but every access crosses the WAN and data isn't in S3.

✓ S3 File Gateway exposing an SMB share — app keeps writing to a 'local' share, objects land in S3, hot files cached on-prem. Access-pattern verbs decide: 'keep using it daily' = Gateway; 'move it' = DataSync.

**Practice Q8.** Thumbnails derived from originals (stored separately) are accessed rarely after 30 days. They can be regenerated at any time. MOST cost-effective class after day 30?

✗ Standard-IA — pays multi-AZ resilience for disposable data.

✗ Glacier Flexible — minutes-to-hours retrieval breaks image serving.

✓ Lifecycle to S3 One Zone-IA — instant access, ~20% cheaper than Standard-IA, and single-AZ risk is acceptable because objects are re-creatable. 'Re-creatable/derived + infrequent' is the One Zone-IA signature.

## Self-check before Module 3b

- Object vs block vs file — one sentence each, plus the AWS product for each.
- Recite the storage-class ladder with retrieval times and minimum durations.
- Why does gp3 beat gp2 for a small high-IOPS volume?
- EFS vs FSx Windows vs FSx Lustre vs FSx ONTAP — trigger phrase for each?
- DataSync vs File Gateway vs Transfer Family vs Snowball — which verb picks which?